DUOC UC · DESARROLLO BACKEND III (PBY2203)

Guion del video Modernización del Backend del Banco XYZ

Qué decir, qué mostrar y qué ejecutar en cada momento · 5 a 7 minutos

Estudiante	[Nombre del estudiante]
Asignatura	Desarrollo Backend III
Evaluación	Evaluación Final Transversal (Semana 9)
Repositorio	github.com/camiluck/banco-xyz-eft
Fecha	Octubre 2026

Duración: 5 a 7 minutos (este guion dura unos 6:30 leyendo a ritmo normal) · **Formato:** MP4 · **Webcam visible todo el video**

Antes de grabar (15 minutos antes)

1. Abre **Docker Desktop** y espera a que diga *Engine running*.

2. En una terminal, dentro de la carpeta del proyecto:

```
docker compose up -d --build
docker compose ps          # espera a que TODOS digan (healthy), 3 a 5 minutos
./scripts/demo-video.sh 1  # prueba rápida: si muestra la tabla, está todo OK
```

> En Windows usa **Git Bash** para los `.sh`. Si no tienes `jq`, instálalo con `winget install jqlang.jq`.

3. Deja abiertas estas **pestañas del navegador**, en este orden:

1. **GitHub**: `github.com/camiluck/banco-xyz-eft` (el readme con el diagrama)
2. **Eureka**: http://localhost:8761
3. **Kafka UI**: http://localhost:8085
4. **GitHub** → `batch-service/src/main/java/cl/bancoxyz/batch/comun/FechaParser.java`
5. **GitHub** → `ms-pagos/src/main/java/cl/bancoxyz/pagos/servicio/PagoService.java`
6. **GitHub** → pestaña **Actions** (la ejecución en verde)

4. Terminal con **letra grande** (Ctrl + `+` varias veces) y ventana maximizada.

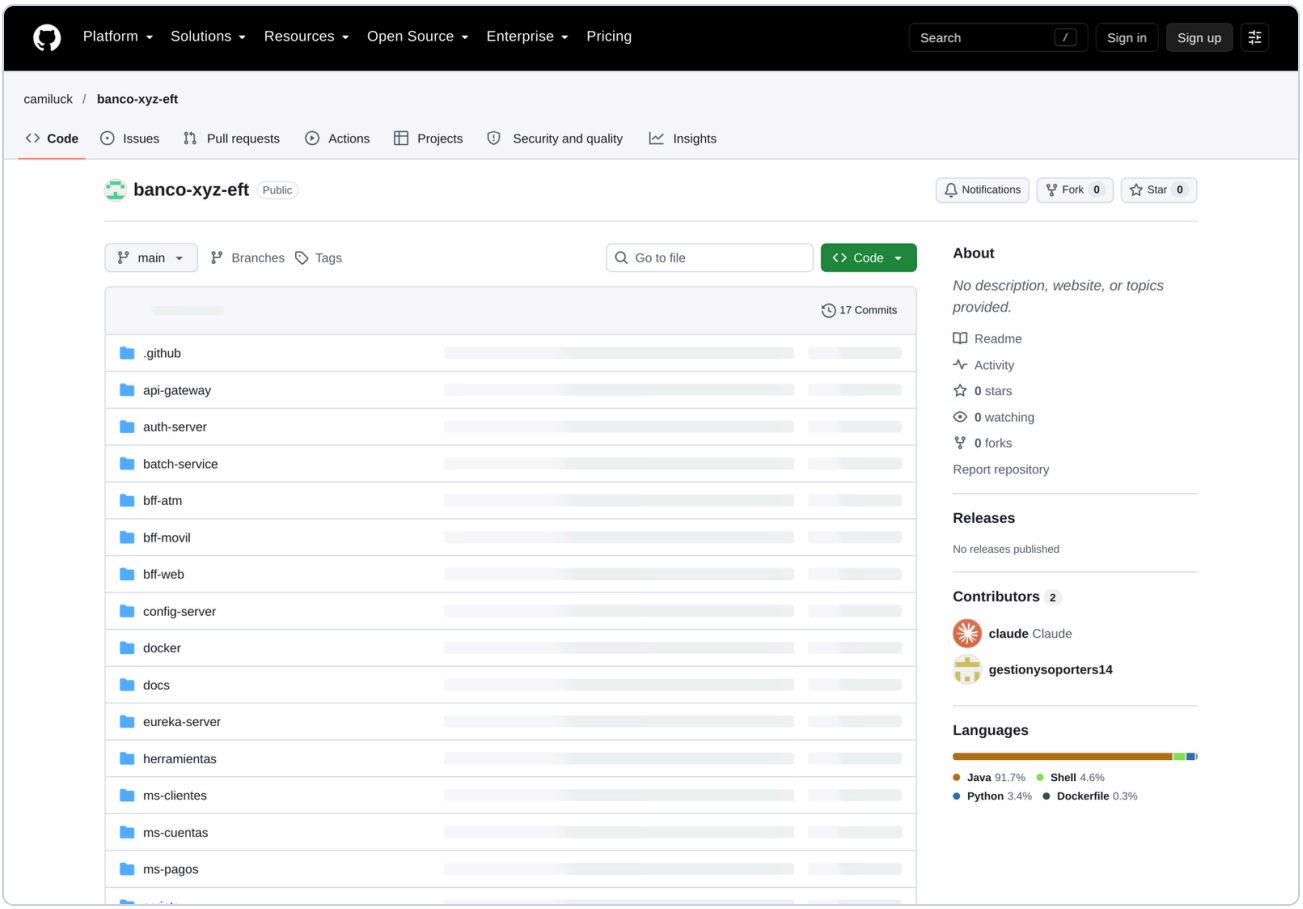
5. En Kaltura, graba **pantalla + webcam** (la cámara en una esquina).

> **Cómo leer este guion.** 🎬 **MOSTRAR** es lo que debe verse en pantalla. 🗣️ **DECIR** es el texto para leer o parafrasear con tus palabras. ⌨️ **EJECUTAR** es el comando. El script `demo-video.sh` muestra cada comando, lo corre y espera ENTER, así que no tienes que tipear nada en vivo.

---

1. Presentación (0:00 – 0:20)

🎬 **MOSTRAR:** pantalla completa con la cámara, o el readme de GitHub de fondo.



Repositorio en GitHub (readme)

🗣️ **DECIR:**

> "Hola, mi nombre es ___. Les presento mi Evaluación Final Transversal de Desarrollo Backend III: la modernización del backend del Banco XYZ, migrando su sistema legacy a una arquitectura de microservicios con Spring Cloud y Spring Batch."

---

2. Resumen ejecutivo (0:20 – 1:20)

🎬 **MOSTRAR:** pestaña 1 (GitHub, readme). Baja lentamente hasta el **diagrama de arquitectura** y, al mencionarlas, señala con el mouse cada capa: canales, BFF, gateway, microservicios, Kafka, base de datos.

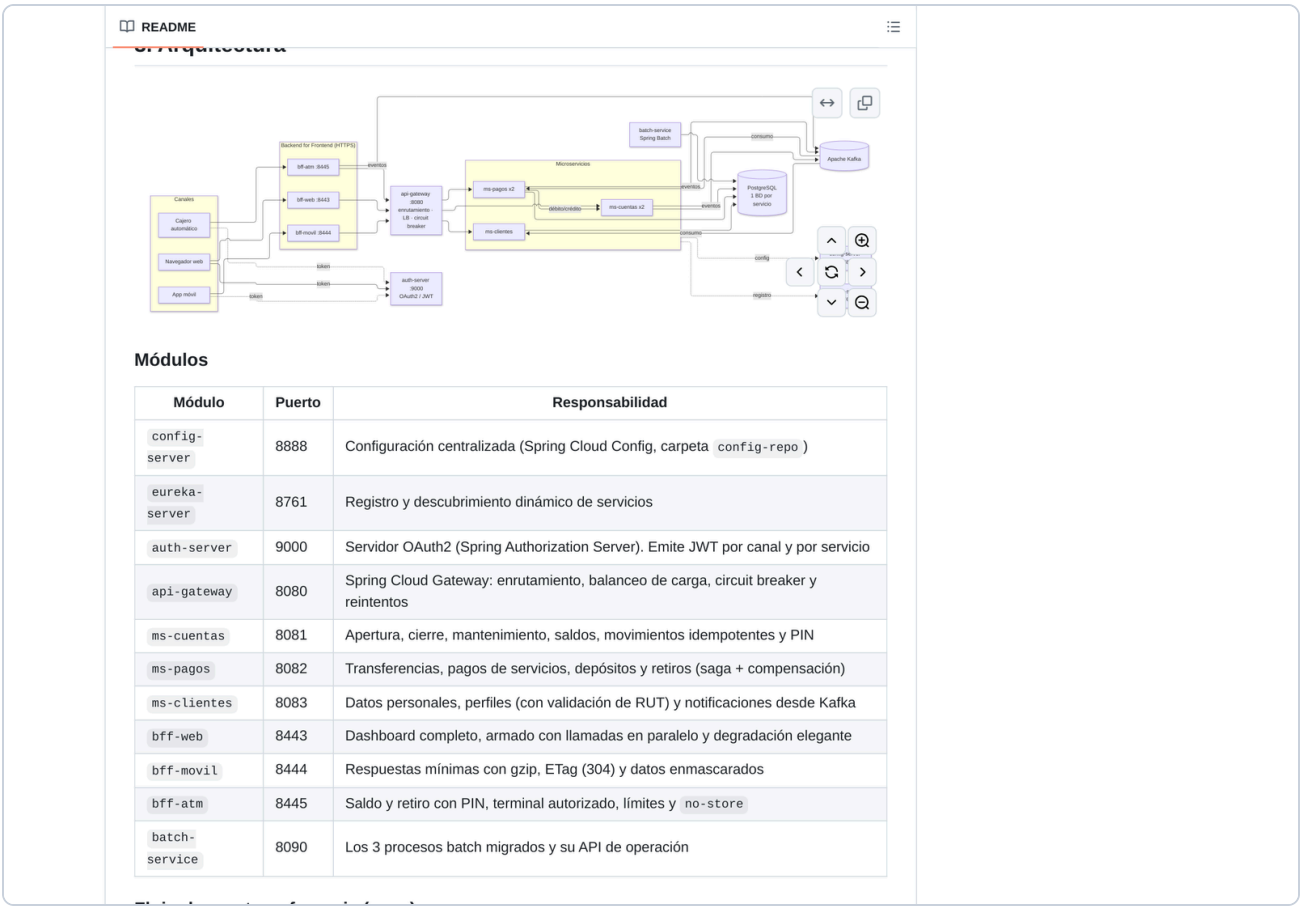


Diagrama de arquitectura en el readme

🗣️ DECIR:

> "El Banco XYZ funcionaba hace más de 30 años sobre COBOL y scripts Shell en un mainframe. Eso le generaba tres problemas: no podía escalar, el mantenimiento era muy caro y era muy difícil integrar canales nuevos.
>
> El objetivo del proyecto fue migrar a una arquitectura de microservicios en la nube. Identifiqué cinco procesos clave: migrar los procesos batch a Spring Batch, dividir el monolito en microservicios, implementar el patrón Backend for Frontend, agregar seguridad distribuida con OAuth2 e integrar mensajería asíncrona con Kafka.
>
> Esto responde a tres requerimientos del negocio: continuidad operativa, o sea que una falla parcial no detenga al banco; escalar a bajo costo; y seguridad e integridad de los datos financieros.
>
> En el diagrama se ve la solución: tres canales, web, móvil y cajeros, cada uno con su propio BFF. Un API Gateway reparte las peticiones entre los microservicios de cuentas, pagos y clientes, que se comunican por eventos en Kafka. Todo se configura con Config Server, se registra en Eureka, se protege con un servidor OAuth2 y corre en Docker."

3. Resultados y comparación con el legacy (1:20 – 4:00)

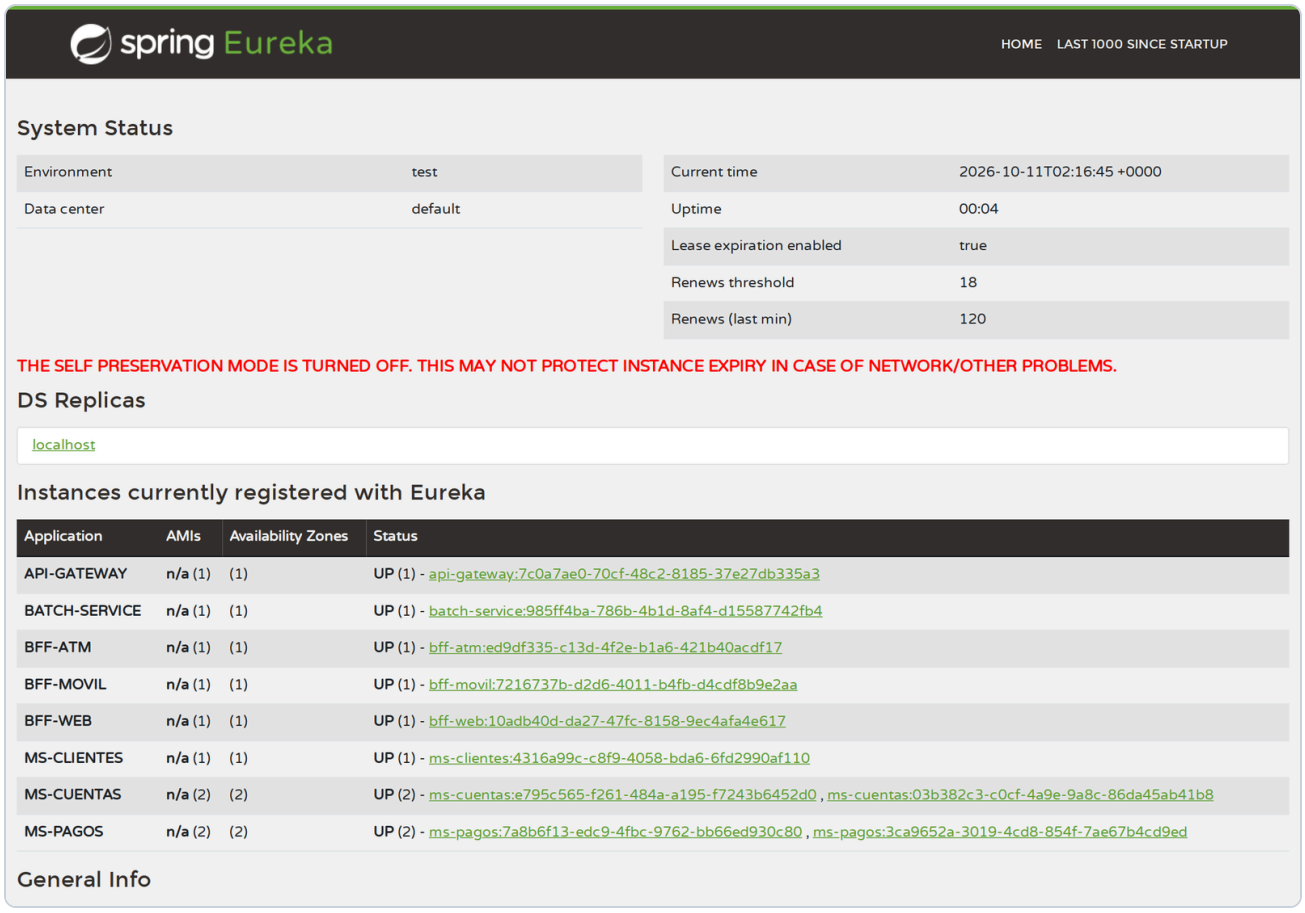
3.1 Infraestructura (1:20 – 1:40)

⌨️ **EJECUTAR:** `./scripts/demo-video.sh 1`

🎬 **MOSTRAR:** la tabla de contenedores, todos *healthy*. Después cambia a la pestaña 2 (**Eureka**) y señala que **MS-CUENTAS** y **MS-PAGOS** tienen **2 instancias** cada uno.



docker compose ps: los 16 contenedores healthy



Eureka: MS-CUENTAS y MS-PAGOS con 2 instancias (UP (2))

🗣️ **DECIR:**

> "Aquí está el sistema corriendo: 16 contenedores. En Eureka se ve que cuentas y pagos tienen dos réplicas cada uno. Eso es escalabilidad horizontal: el gateway balancea la carga entre ellas, y puedo agregar más réplicas con un solo comando, sin tocar el código."

3.2 Procesos batch (1:40 – 2:40)

⌨️ **EJECUTAR:** `./scripts/demo-video.sh 2` y presiona ENTER entre pantallas.

🎬 **MOSTRAR, en orden:**

1. El job lanzado y los puntitos de "Procesando".
2. **Pasos y particiones**: señala las columnas *leidos*, *escritos* y *omitidos*, y la salida `COMPLETADO_CON_OMISIONES`.
3. **Rechazos agrupados por motivo** (MONTO_VACIO, FECHA_INVALIDA, etc.) y el detalle con línea y contenido.
4. El **resumen diario**.
5. La **falla simulada**: las líneas `[SIMULACION] Falla transitoria`, los rollbacks y el estado final `COMPLETED`.

Terminal — banco-xyz-eft

```
~/banco-xyz-eft $ curl -X POST http://localhost:8090/api/batch/jobs/transaccionesDiariasJob?dataset=semana_3
{
  "consultar": "/api/batch/ejecuciones/1",
  "estado": "STARTING",
  "instanciaId": 1,
  "job": "transaccionesDiariasJob",
  "ejecucionId": 1
}
Procesando.
```

1) Se lanza el job de transacciones diarias

Terminal — banco-xyz-eft

```
Resultado por paso y por partición (leídos / escritos / omitidos)
~/banco-xyz-eft $ curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8090/api/batch/ejecuciones/1 | jq '{estado,
salida, pasos: [.pasos[] | {paso, salida, leidos, escritos, omitidos, commits}]}'
{
  "estado": "COMPLETED",
  "salida": "COMPLETED",
  "pasos": [
    {
      "paso": "transaccionesArchivoStep:partition0",
      "salida": "COMPLETED",
      "leidos": 1000,
      "escritos": 789,
      "omitidos": 211,
      "commits": 11
    },
    {
      "paso": "transaccionesCargaStep",
      "salida": "COMPLETADO_CON_OMISIONES",
      "leidos": 1000,
      "escritos": 789,
      "omitidos": 211,
      "commits": 11
    },
    {
      "paso": "transaccionesResumenStep",
      "salida": "COMPLETED",
      "leidos": 0,
      "escritos": 325,
      "omitidos": 0,
      "commits": 1
    }
  ]
}
```

2) Resultado por paso: 1000 leídos, 789 escritos, 211 omitidos, COMPLETADO_CON_OMISIONES

Terminal — banco-xyz-eft

```
Registros rechazados: cada uno queda auditado con su motivo (el legacy fallaba en silencio)
~/banco-xyz-eft $ curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8090/api/batch/resultados/1/rechazos | jq
[
  {
    "motivo": "MONTO_VACIO",
    "cantidad": 162
  },
  {
    "motivo": "FECHA_INVALIDA",
    "cantidad": 49
  }
]

~/banco-xyz-eft $ curl -s -H "Authorization: Bearer $TOKEN" "http://localhost:8090/api/batch/resultados/1/rechazos?detall
e=true" | jq '.[0:3]'
[
  {
    "archivo": "semana_3/movimientos_financieros_diarios.csv",
    "linea": 6,
    "contenido": "5,17-08-2024,,debito",
    "motivo": "MONTO_VACIO: el campo monto está vacío"
  },
  {
    "archivo": "semana_3/movimientos_financieros_diarios.csv",
    "linea": 7,
    "contenido": "6,2024-09-26,,invalido",
    "motivo": "MONTO_VACIO: el campo monto está vacío"
  },
  …
```

3) Rechazos agrupados por motivo y detalle con línea y contenido

Terminal — banco-xyz-eft

```
Resumen diario generado (primeros 5 días)
~/banco-xyz-eft $ curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8090/api/batch/resultados/1/transacciones |
jq '.[0:5]'
[
  {
    "instancia_id": 1,
    "fecha": "2024-01-01",
    "total_registros": 4,
    "cantidad_creditos": 1,
    "cantidad_debitos": 1,
    "monto_creditos": 1000.00,
    "monto_debitos": 3000.00,
    "saldo_neto": -2000.00,
    "anomalias": 2
  },
  {
    "instancia_id": 1,
    "fecha": "2024-01-02",
    "total_registros": 1,
    "cantidad_creditos": 0,
    "cantidad_debitos": 1,
    "monto_creditos": 0.00,
    "monto_debitos": 3000.00,
    "saldo_neto": -3000.00,
    "anomalias": 0
  },
  …
```

4) Resumen diario generado



5) Falla simulada: 2 fallas, reintentos (rollbacks=2) y COMPLETED

🗣️ **DECIR:**

> "Esta es la Parte 1. Migré los tres procesos batch: transacciones diarias, intereses mensuales y estados de cuenta anuales. Cada uno lee los archivos del legacy, valida, normaliza y escribe en chunks transaccionales.
>
> Los datos vienen con errores a propósito: cuatro formatos de fecha, montos vacíos o negativos, duplicados. El sistema legacy fallaba en silencio. Acá cada registro rechazado queda auditado con su línea y su motivo, y el job termina con el estado 'completado con omisiones'.
>
> Además, cada archivo es una partición y se procesan en paralelo.
>
> Ahora simulo que se cae la conexión a la base de datos en medio del proceso. Spring Batch reintenta con espera exponencial y el job termina bien, sin duplicar datos. Si la falla es crítica, el job se reinicia automáticamente desde el último commit."

🎬 **MOSTRAR (5 segundos):** pestaña 1, sección **"Equivalencia con el legacy"** del readme (la tabla).

README

forma independiente. Los tests `EquivalenciaLegacyTests` comprueban que Spring Batch obtiene exactamente los mismos resultados:

Dataset `semana_3`	Referencia legacy	Spring Batch
Transacciones: guardadas / anomalías / omitidas	789 / 397 / 211	789 / 397 / 211
Total créditos / débitos válidos	244.500 / 284.900	244.500 / 284.900
Intereses: cuentas calculadas / omitidas / interés total	340 / 660 / 15.440,50	340 / 660 / 15.440,50
Estados: movimientos / omitidos / cuentas / saldo neto	759 / 241 / 20 / −418.700	759 / 241 / 20 / −418.700

5. Comparación con el sistema legacy

Aspecto	Legacy (COBOL + Shell, mainframe)	Nueva arquitectura
Escalabilidad	Vertical (hardware más grande)	Horizontal: réplicas en contenedores y particiones batch en paralelo
Fallos	Un módulo caído detiene el sistema	Aislamiento por servicio, circuit breaker, fallback y reintentos
Batch	Si falla, se vuelve a correr desde cero	Restart desde el último *commit*, retry automático y skip auditado
Calidad de datos	Errores silenciosos	Cada registro rechazado queda con línea, contenido y motivo
Seguridad	Centralizada y perimetral	OAuth2/JWT en cada servicio, token por canal, HTTPS y PIN con BCrypt
Canales	Misma respuesta para todos	Un BFF por canal, con su propia optimización y seguridad
Integración	Síncrona y acoplada	Eventos Kafka en tiempo real, consumidores idempotentes y DLT
Despliegue	Ventanas de mantenimiento	Contenedores Docker, CI en GitHub Actions y despliegue en AWS
Monitoreo	Logs dispersos	Actuator, Prometheus, traceId en cada log y Kafka UI

6. Tecnologías

Tabla de equivalencia con el legacy en el readme

🗣️ **DECIR:**

> "Para asegurar que los resultados son equivalentes al legacy, implementé las mismas reglas de forma independiente y las pruebas automáticas comparan los totales: coinciden exactamente."

3.3 Backend for Frontend (2:40 – 3:10)

⌨️ **EJECUTAR:** `./scripts/demo-video.sh 3`

🎬 **MOSTRAR:** la respuesta del móvil (corta, con la cuenta enmascarada `****0001`), luego el resumen web y, al final, la **comparación de bytes** WEB vs MOVIL.

```
Terminal — banco-xyz-eft

~/banco-xyz-eft $ curl -sk -H "Authorization: Bearer $T_MOVIL" https://localhost:8444/movil/clientes/1/inicio | jq
{
  "nombre": "Juan",
  "saldoTotal": 1850000.00,
  "cuentas": [
    {
      "id": 1,
      "tipo": "AHORRO",
      "num": "****0001",
      "saldo": 1500000.00
    },
    {
      "id": 2,
      "tipo": "CORRIENTE",
      "num": "****0002",
      "saldo": 350000.00
    }
  ],
  "ultimas": [
    {
      "f": "2026-10-11",
      "desc": "Saldo migrado desde sistema legacy",
      "monto": 1500000.00,
      "signo": "+"
    }
  ]
}
```

Respuesta del BFF móvil: corta y con la cuenta enmascarada

```
Terminal — banco-xyz-eft

~/banco-xyz-eft $ curl -sk -H "Authorization: Bearer $T_WEB" https://localhost:8443/web/clientes/1/resumen | jq '{client
e: .cliente.nombres, saldoTotal, cuentas: (.cuentas|length), operaciones: (.operacionesRecientes|length), notificaciones:
(.notificaciones|length)}'
{
  "cliente": "Juan Andrés",
  "saldoTotal": 1850000.00,
  "cuentas": 2,
  "operaciones": 0,
  "notificaciones": 0
}

Tamaño de la respuesta de cada canal
~/banco-xyz-eft $ curl -sk -o /dev/null -w 'WEB:   %{size_download} bytes\n' -H "Authorization: Bearer $T_WEB" https://lo
calhost:8443/web/clientes/1/resumen
WEB:   1188 bytes

~/banco-xyz-eft $ curl -sk -o /dev/null -w 'MOVIL: %{size_download} bytes\n' -H "Authorization: Bearer $T_MOVIL" https://
localhost:8444/movil/clientes/1/inicio
MOVIL: 283 bytes
```

Resumen web y comparación de tamaño: WEB vs MÓVIL

🗣️ **DECIR:**

> "Parte 2: el patrón Backend for Frontend. Antes, todos los canales recibían la misma respuesta pesada del monolito. Ahora el BFF móvil entrega sólo lo esencial: nombres cortos, número de cuenta enmascarado, compresión gzip y ETag. El BFF web entrega el dashboard completo y arma las consultas en paralelo. Miren la diferencia de tamaño entre las dos respuestas."

3.4 Seguridad (3:10 – 3:35)

⌨️ **EJECUTAR:** `./scripts/demo-video.sh 4`

🎬 **MOSTRAR:** el `HTTP 401`, el `TERMINAL_NO_AUTORIZADO`, el `PIN_INCORRECTO` y los **dos retiros con el mismo comprobante**.

```
Terminal — banco-xyz-eft

Token del canal WEB usado en el BFF MÓVIL:
~/banco-xyz-eft $ curl -sk -o /dev/null -w 'HTTP %{http_code}\n' -H "Authorization: Bearer $T_WEB" https://localhost:844
4/movil/cuentas/1/saldo
HTTP 401

Cajero desde un terminal NO registrado:
~/banco-xyz-eft $ curl -sk -H "Authorization: Bearer $T_ATM" -H 'X-Terminal-Id: ATM-FALSO' -H 'Content-Type: application/
json' -d '{"cuentaId":1,"pin":"1234"}' https://localhost:8445/atm/consulta-saldo; echo
{"title":"TERMINAL_NO_AUTORIZADO","codigo":"TERMINAL_NO_AUTORIZADO","status":403}

PIN incorrecto:
~/banco-xyz-eft $ curl -sk -H "Authorization: Bearer $T_ATM" -H 'X-Terminal-Id: ATM-STGO-001' -H 'Content-Type: applicati
on/json' -d '{"cuentaId":3,"pin":"9999"}' https://localhost:8445/atm/consulta-saldo | jq
{
  "type": "about:blank",
  "title": "PIN_INCORRECTO",
  "status": 401,
  "detail": "PIN incorrecto. Intentos restantes: 2",
  "instance": "/atm/consulta-saldo",
  "codigo": "PIN_INCORRECTO"
}
```

Token de otro canal (401), terminal falso (403) y PIN incorrecto (401)

```
Terminal — banco-xyz-eft

Retiro en cajero, y el MISMO retiro reenviado (corte de red): no se cobra dos veces
~/banco-xyz-eft $ curl -sk -H "Authorization: Bearer $T_ATM" -H 'X-Terminal-Id: ATM-STGO-001' -H 'Idempotency-Key: video-
26173' -H 'Content-Type: application/json' -d '{"cuentaId":1,"pin":"1234","monto":20000}' https://localhost:8445/atm/reti
ros | jq
{
  "comprobante": "ATM-ATM-STGO-001-video-26173",
  "estado": "COMPLETADO",
  "montoEntregado": 20000,
  "saldoDisponible": 1480000.00,
  "mensaje": "Retire su dinero"
}

~/banco-xyz-eft $ curl -sk -H "Authorization: Bearer $T_ATM" -H 'X-Terminal-Id: ATM-STGO-001' -H 'Idempotency-Key: video-
26173' -H 'Content-Type: application/json' -d '{"cuentaId":1,"pin":"1234","monto":20000}' https://localhost:8445/atm/reti
ros | jq
{
  "comprobante": "ATM-ATM-STGO-001-video-26173",
  "estado": "COMPLETADO",
  "montoEntregado": 20000.00,
  "saldoDisponible": 1480000.00,
  "mensaje": "Retire su dinero"
}
```

El mismo retiro enviado 2 veces: mismo comprobante y mismo saldo

🗣️ **DECIR:**

> "Cada canal tiene su propia autenticación con OAuth2 y JWT. Un token del canal web no sirve en la app móvil: responde 401. El cajero, además del token, exige un terminal registrado y el PIN, que se bloquea a los tres intentos fallidos. Todo viaja por HTTPS.
>
> Y si el cajero pierde la conexión y reenvía el retiro, gracias a la clave de idempotencia no se cobra dos veces: mismo comprobante, mismo saldo."

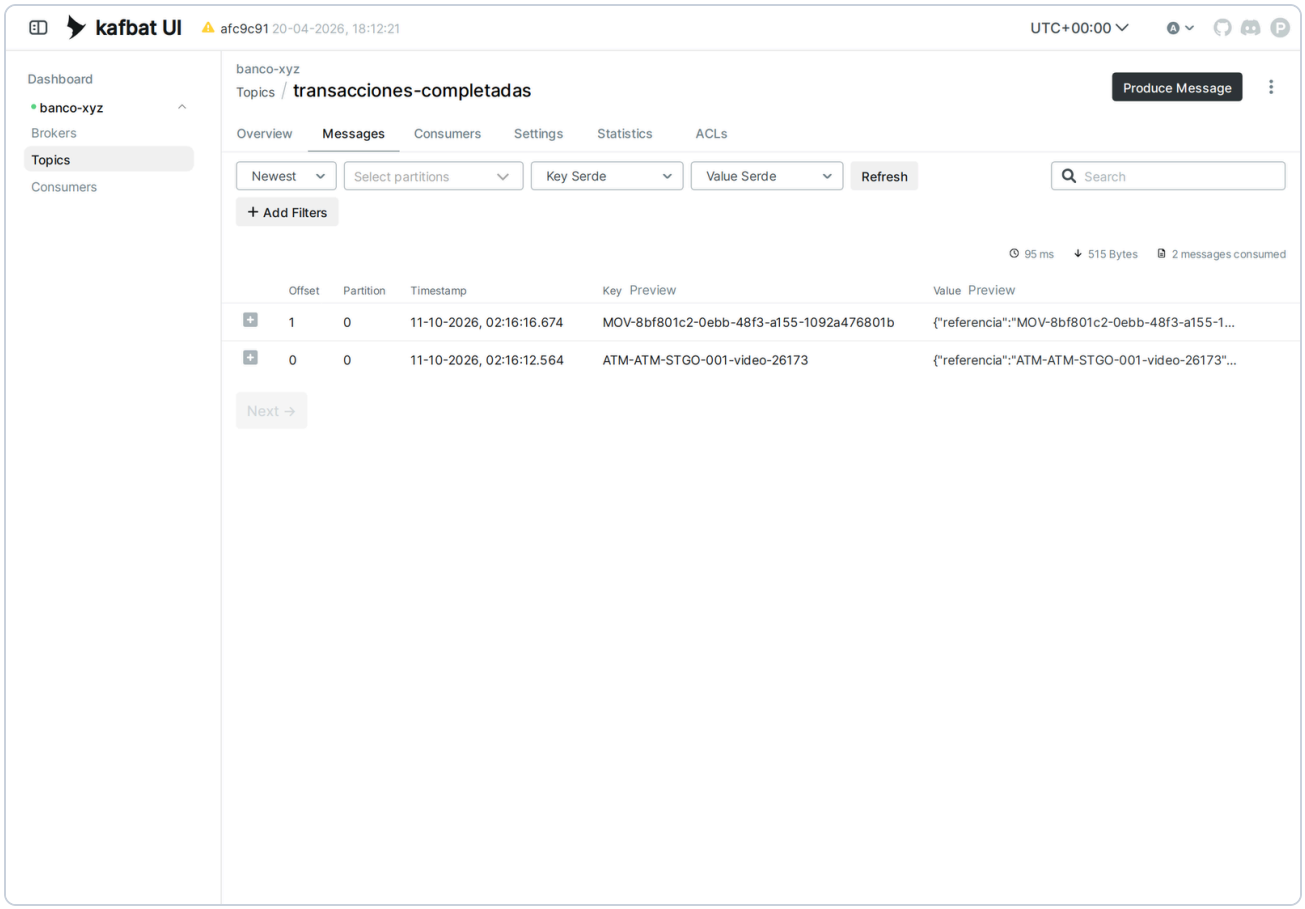
3.5 Kafka (3:35 – 3:50)

⌨️ **EJECUTAR:** `./scripts/demo-video.sh 5`

🎬 **MOSTRAR:** la transferencia `COMPLETADO` y el aviso que recibió el cliente 2. Después ve a la pestaña 3 (**Kafka UI**) → *Topics* → `transacciones-completadas` → *Messages* y abre el último mensaje.



Transferencia completada y aviso recibido por el cliente 2



Kafka UI: mensajes del tópico transacciones-completadas

🗣️ **DECIR:**

> "Parte 3: los microservicios se comunican por eventos. Al completar la transferencia, pagos publica un evento en Kafka y el microservicio de clientes lo consume en tiempo real para notificar a la persona que recibió el dinero. Aquí se ve el mensaje en el tópico."

3.6 Comparación con el legacy (3:50 – 4:05)

🎬 **MOSTRAR:** pestaña 1, tabla **"Comparación con el sistema legacy"** del readme.

README

Aspecto	Legacy (COBOL + Shell, mainframe)	Nueva arquitectura
Escalabilidad	Vertical (hardware más grande)	Horizontal: réplicas en contenedores y particiones batch en paralelo
Fallos	Un módulo caído detiene el sistema	Aislamiento por servicio, circuit breaker, fallback y reintentos
Batch	Si falla, se vuelve a correr desde cero	Restart desde el último *commit*, retry automático y skip auditado
Calidad de datos	Errores silenciosos	Cada registro rechazado queda con línea, contenido y motivo
Seguridad	Centralizada y perimetral	OAuth2/JWT en cada servicio, token por canal, HTTPS y PIN con BCrypt
Canales	Misma respuesta para todos	Un BFF por canal, con su propia optimización y seguridad
Integración	Síncrona y acoplada	Eventos Kafka en tiempo real, consumidores idempotentes y DLT
Despliegue	Ventanas de mantenimiento	Contenedores Docker, CI en GitHub Actions y despliegue en AWS
Monitoreo	Logs dispersos	Actuator, Prometheus, traceId en cada log y Kafka UI

6. Tecnologías

Java 21 · Spring Boot 3.5 · Spring Cloud 2025.0 (Config, Netflix Eureka, Gateway, LoadBalancer, Circuit Breaker) · Spring Security OAuth2 (Authorization Server y Resource Server) · Spring Batch 5 · Spring Data JPA + Flyway · Resilience4j · Apache Kafka 3.9 · PostgreSQL 16 / H2 · Micrometer (Prometheus y tracing) · Docker y Docker Compose · GitHub Actions.

7. Estructura del repositorio

```
├── config-server/      config-repo/ con la configuración de todos los servicios
├── eureka-server/
├── auth-server/
├── api-gateway/
```

Tabla de comparación con el legacy en el readme

🗣️ **DECIR:**

> "En resumen, frente al legacy: pasamos de escalar en vertical a escalar en horizontal; de un sistema donde un error lo botaba todo, a servicios aislados con circuit breaker; de procesos batch que había que repetir desde cero, a reinicios desde el último punto; y de seguridad perimetral, a un token validado en cada servicio y por canal."

---

4. Desafíos y soluciones (4:05 – 5:35)

Desafío 1: datos legacy con errores (4:05 – 4:30)

🎬 **MOSTRAR:** pestaña 4 (**FechaParser.java**) y señala la lista de formatos de fecha.

batch-service › FechaParser.java

```java
/**
 * Normaliza las fechas del sistema legacy, que vienen en 4 formatos distintos:
 * yyyy-MM-dd (estándar), yyyy/MM/dd, dd-MM-yyyy y dd/MM/yyyy (formato chileno: día primero).
 * La validación es estricta: 2024-02-30 se rechaza.
 */
public final class FechaParser {

    private static final DateTimeFormatter ISO = DateTimeFormatter.ofPattern("uuuu-MM-dd").withResolverStyle(ResolverStyle.STRICT);
    private static final List<DateTimeFormatter> ALTERNATIVOS = List.of(
            DateTimeFormatter.ofPattern("uuuu/MM/dd").withResolverStyle(ResolverStyle.STRICT),
            DateTimeFormatter.ofPattern("dd-MM-uuuu").withResolverStyle(ResolverStyle.STRICT),
            DateTimeFormatter.ofPattern("dd/MM/uuuu").withResolverStyle(ResolverStyle.STRICT));

    private FechaParser() {
    }

    /** @param formatoCorregido true si la fecha venía en un formato no estándar y fue normalizada */
    public record Resultado(LocalDate fecha, boolean formatoCorregido) {
    }

    public static Resultado parsear(String texto) {
        if (texto == null || texto.isBlank()) {
            throw new RegistroInvalidoException("FECHA_VACIA", "la fecha es obligatoria");
        }
        String t = texto.trim();
        try {
            return new Resultado(LocalDate.parse(t, ISO), false);
        } catch (DateTimeParseException ignorada) {
            // se prueban los formatos alternativos
        }
        for (DateTimeFormatter f : ALTERNATIVOS) {
            try {
                return new Resultado(LocalDate.parse(t, f), true);
            } catch (DateTimeParseException ignorada) {
                // siguiente formato
            }
        }
        throw new RegistroInvalidoException("FECHA_INVALIDA", "formato de fecha no reconocido: " + t);
    }
}
```

FechaParser.java: los 4 formatos de fecha del legacy

🗣️ **DECIR:**

> "El primer desafío fueron los datos legacy. Las fechas venían en cuatro formatos distintos, había montos vacíos, negativos y registros duplicados. La solución fue leer todo como texto y separar tres casos: lo que se puede corregir, como las fechas, se normaliza; lo sospechoso, como un monto negativo, se guarda marcado como anomalía; y lo inválido se omite, pero queda auditado."

Desafío 2: transferencias entre microservicios (4:30 – 5:00)

🎬 **MOSTRAR:** pestaña 5 (**PagoService.java**) y baja hasta el método `ejecutar` y el método `compensar`.

☕ ms-pagos › PagoService.java — ejecutar()

```java
    /** Ejecuta (o retoma) los pasos pendientes de un pago. Seguro de llamar varias veces. */
    public Pago ejecutar(Pago pago) {
        if (pago.getEstado() != EstadoPago.PENDIENTE) {
            return pago;
        }
        pago.registrarIntento();
        String ref = pago.getReferencia();
        String desc = descripcion(pago);
        try {
            if (pago.faltaDebito()) {
                MovimientoCuenta debito = cuentas.aplicarMovimiento(pago.getCuentaOrigenId(), "DEBITO",
                        pago.getMonto(), ref + "-D", desc);
                pago.marcarDebito(debito.clienteId());
                pago = pagos.save(pago);
            }
            if (pago.faltaCredito()) {
                try {
                    MovimientoCuenta credito = cuentas.aplicarMovimiento(pago.getCuentaDestinoId(), "CREDITO",
                            pago.getMonto(), ref + "-C", desc);
                    pago.marcarCredito(credito.clienteId());
                } catch (HttpClientErrorException rechazoDestino) {
                    if (pago.isDebitoAplicado()) {
                        return compensar(pago, motivo(rechazoDestino));
                    }
                    throw rechazoDestino;
                }
            }
            pago.completar();
            pago = pagos.save(pago);
            log.info("Pago {} COMPLETADO", ref);
            publicarCompletado(pago);
            return pago;
        } catch (HttpClientErrorException rechazo) {
            pago.rechazar(motivo(rechazo));
            log.info("Pago {} RECHAZADO: {}", ref, pago.getMotivo());
            return pagos.save(pago);
        } catch (CuentasNoDisponibleException caido) {
            if (!pago.isDebitoAplicado() && pago.getIntentos() >= maxIntentos) {
                pago.rechazar("Servicio de cuentas no disponible tras " + pago.getIntentos() + " intentos");
                log.warn("Pago {} RECHAZADO por indisponibilidad prolongada", ref);
            } else {
                pago.anotar("En proceso: servicio de cuentas no disponible, se reintentará automáticamente");
                log.warn("Pago {} queda PENDIENTE (intento {})", ref, pago.getIntentos());
            }
            return pagos.save(pago);
        }
    }
```

PagoService.ejecutar(): débito, crédito y compensación si el destino rechaza

☕ ms-pagos › PagoService.java — compensar()

```java
    /** Transacción compensatoria: devuelve el dinero debitado a la cuenta origen. */
    private Pago compensar(Pago pago, String motivo) {
        log.warn("Destino rechazó el crédito de {} ({}). Compensando débito...", pago.getReferencia(), motivo);
        cuentas.aplicarMovimiento(pago.getCuentaOrigenId(), "CREDITO", pago.getMonto(),
                pago.getReferencia() + "-R", "Reverso: " + motivo);
        pago.revertir("Revertido: " + motivo);
        return pagos.save(pago);
    }

    private void publicarCompletado(Pago p) {
```

PagoService.compensar(): devuelve el dinero a la cuenta origen

🗣️ **DECIR:**

> "El segundo desafío: una transferencia toca dos cuentas, pero ya no hay una sola base de datos. Lo resolví con el patrón Saga: primero se debita, luego se acredita, y si el destino rechaza, se ejecuta una compensación que devuelve el dinero. Cada movimiento lleva una referencia única, así que un reintento nunca descuenta dos veces. Si el servicio de cuentas está caído, el pago queda pendiente y se completa solo cuando el servicio vuelve."

Desafío 3: resiliencia, en vivo (5:00 – 5:35)

⌨️ **EJECUTAR:** `./scripts/demo-video.sh 6`

🎬 **MOSTRAR:** `docker compose stop ms-clientes`, luego el **503 SERVICIO_NO_DISPONIBLE** en web, el `datosParciales: true` en móvil y la transferencia que **sí funciona**.

Terminal — banco-xyz-eft

```
~/banco-xyz-eft $ docker compose stop ms-clientes
 Container banco-xyz-ms-clientes-1  Stopping
 Container banco-xyz-ms-clientes-1  Stopped

Web: el perfil es esencial → error controlado e inmediato (no un timeout):
~/banco-xyz-eft $ curl -sk -H "Authorization: Bearer $T_WEB" https://localhost:8443/web/clientes/1/resumen | jq
{
  "type": "about:blank",
  "title": "SERVICIO_NO_DISPONIBLE",
  "status": 503,
  "detail": "El servicio de clientes no está disponible. Intente nuevamente en unos minutos.",
  "instance": "/web/clientes/1/resumen",
  "codigo": "SERVICIO_NO_DISPONIBLE"
}
```

Se apaga ms-clientes: la web responde 503 controlado

Terminal — banco-xyz-eft

```
Móvil: la app sigue funcionando con datos parciales:
~/banco-xyz-eft $ curl -sk -H "Authorization: Bearer $T_MOVIL" https://localhost:8444/movil/clientes/1/inicio | jq
{
  "saldoTotal": 1815000.00,
  "cuentas": [
    {
      "id": 1,
      "tipo": "AHORRO",
      "num": "****0001",
      "saldo": 1465000.00
    },
    {
      "id": 2,
      "tipo": "CORRIENTE",
      "num": "****0002",
      "saldo": 350000.00
    }
  ],
  "ultimas": [
    {
      "f": "2026-10-11",
      "desc": "Transferencia: Demo video",
  …

Las transferencias siguen funcionando (no dependen de clientes):
~/banco-xyz-eft $ curl -sk -H "Authorization: Bearer $T_MOVIL" -H 'Content-Type: application/json' -d '{"origen":1,"destino":3,"monto":1000}' https://localhost:8444/movil/transferencias | jq
{
  "ref": "MOV-6b8446f6-a276-4717-a6c2-82d9dd2d10c7",
  "estado": "COMPLETADO",
  "msg": "Transferencia realizada"
}
```

El móvil sigue con datos parciales y la transferencia funciona

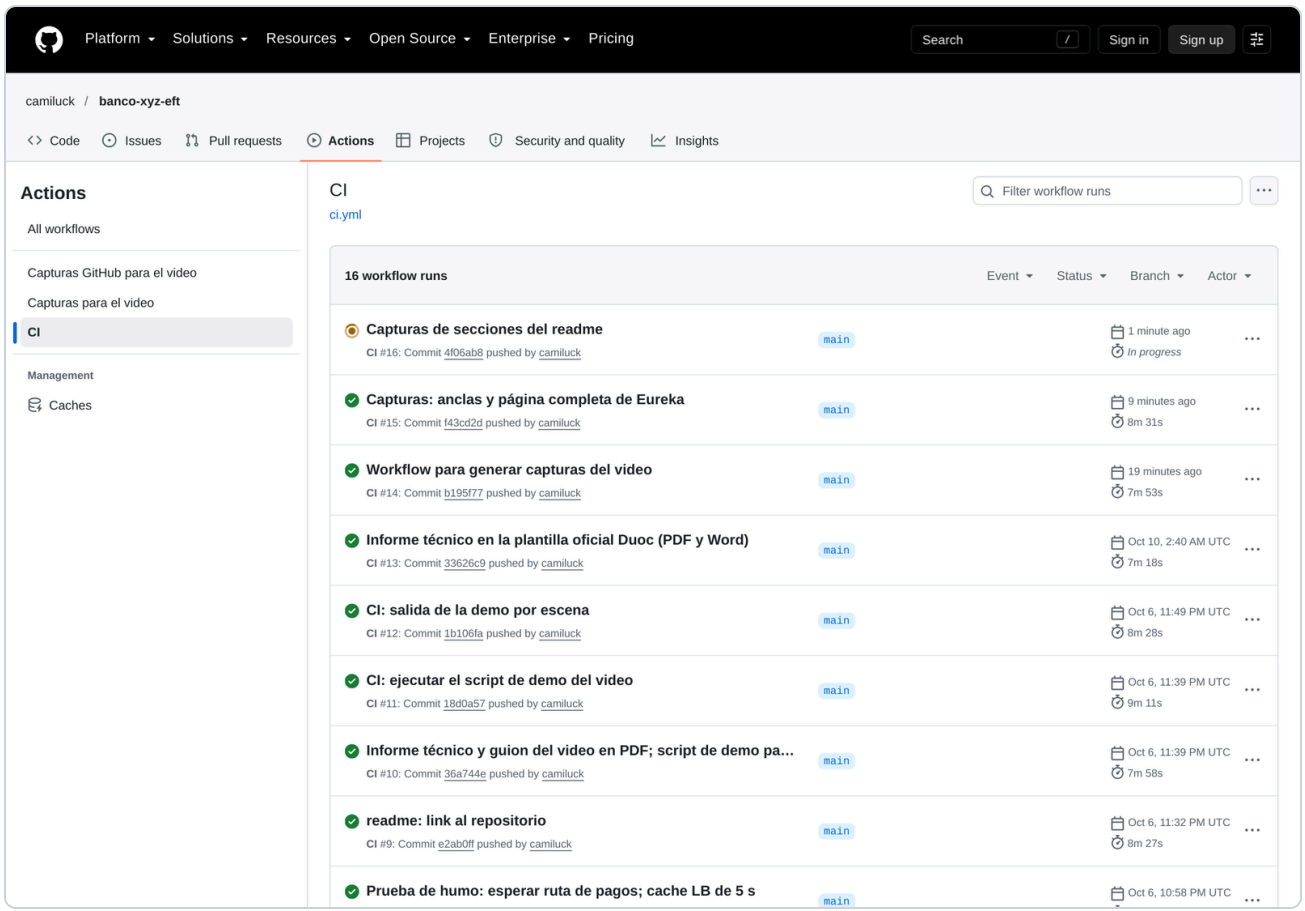
🗣️ **DECIR:**

> "El tercer desafío fue que una falla no botara todo el sistema. Voy a apagar en vivo el microservicio de clientes. La web responde de inmediato con un error controlado, no con un timeout, gracias al circuit breaker de Resilience4j. La app móvil sigue funcionando con datos parciales. Y las transferencias siguen operando, porque no dependen de ese servicio. Cuando el servicio vuelve, el circuito se cierra solo."

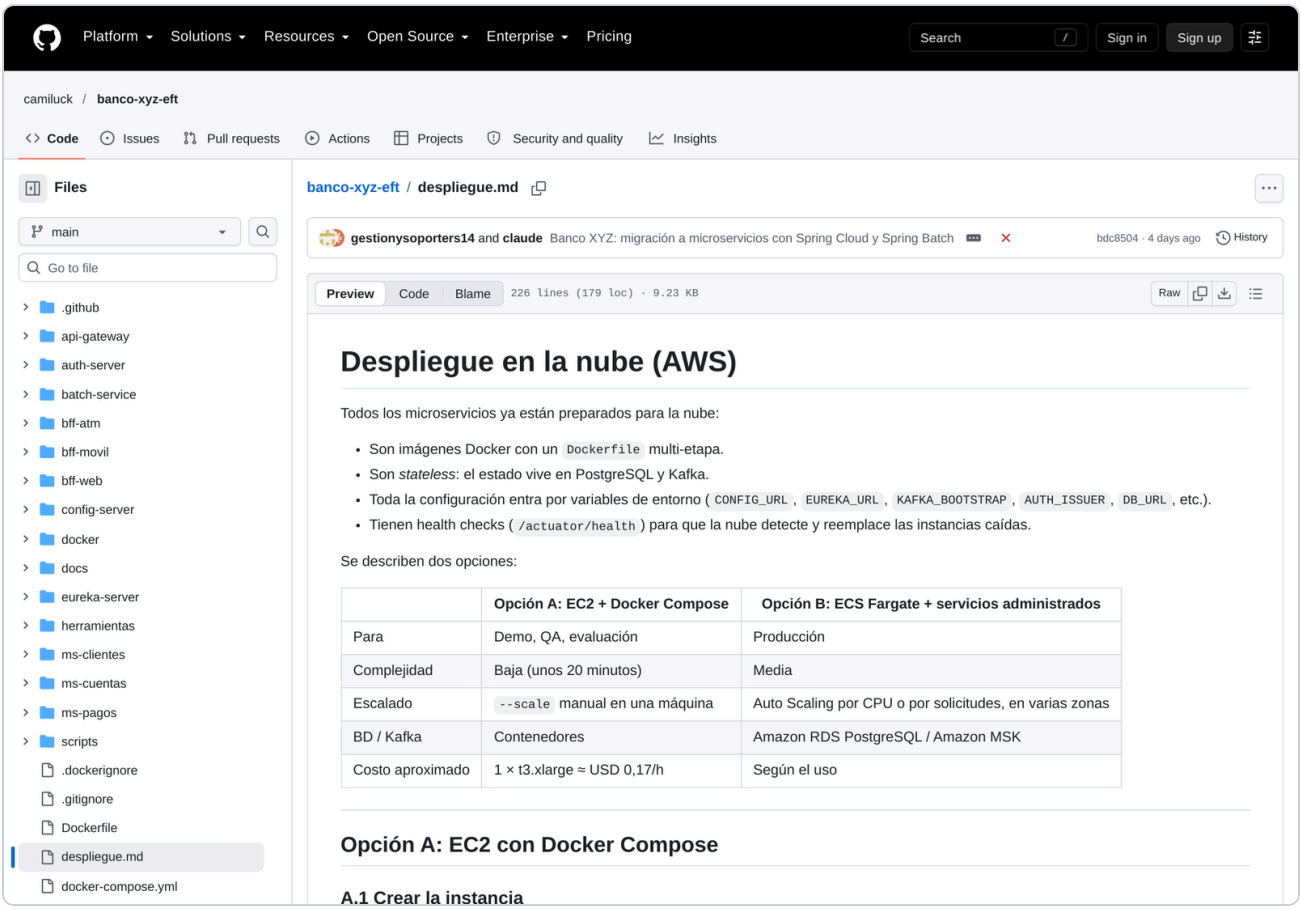
---

5. Propuestas de mejora y próximos pasos (5:35 – 6:15)

🎬 **MOSTRAR:** pestaña 6 (**GitHub Actions** en verde). Al hablar del despliegue, abre `despliegue.md` en GitHub.



GitHub Actions: todas las ejecuciones en verde



despliegue.md: guía para AWS

🗣️ **DECIR:**

> "El proyecto tiene integración continua: en cada cambio, GitHub compila, ejecuta las pruebas y levanta el sistema completo con Docker para probarlo de punta a punta.
>
> Como próximos pasos propongo: primero, el patrón Outbox, para garantizar que los eventos se publiquen aunque Kafka esté caído en ese momento. Segundo, integrar un proveedor de identidad corporativo con autenticación de dos factores y rotación de llaves. Tercero, limitar la cantidad de peticiones por cliente en el gateway. Cuarto, observabilidad con Grafana y Prometheus. Y quinto, desplegar en AWS con ECS Fargate, RDS y MSK, con autoescalado, como dejé documentado en despliegue.md."

---

6. Cierre (6:15 – 6:30)

🎬 **MOSTRAR:** cámara, o el readme.

🗣️ **DECIR:**

> "El código, las instrucciones para ejecutar cada componente, la guía de despliegue y el informe técnico están en el repositorio. Muchas gracias."

Checklist rápido de la pauta (para no olvidar nada)

Punto que pide la pauta	Dónde está en el video
Resumen ejecutivo: objetivos, alcance, características	Sección 2
Resultados obtenidos y comparación con el legacy	Sección 3 (3.1 a 3.6)
Desafíos enfrentados y soluciones implementadas	Sección 4
Propuestas de mejora y próximos pasos	Sección 5
Evidencias del trabajo	Terminal, Eureka, Kafka UI, código y GitHub Actions
Webcam	Todo el video
Duración de 5 a 7 minutos	Unos 6:30

Si algo sale mal durante la grabación

Problema	Solución
Un comando muestra `503` justo al empezar	Espera 1 minuto (los servicios se están registrando) y repite la escena: `./scripts/demo-video.sh N`
"No se pudo obtener token"	`docker compose ps` : algún servicio no está *healthy*. Ejecuta `docker compose up -d` y espera
Te equivocaste hablando	No pares: Kaltura permite recortar, o graba por secciones y únelas
Te pasaste de 7 minutos	Acorta la escena 3.2 (muestra sólo los rechazos y la falla simulada)